

CiteScout

Discovery i archiwizacja prac naukowych przez cytowania

Raport projektowy — REST API

Krzysztof Czuba

24.06.2026

Spis treści

1	Opis funkcjonalności	3
1.1	Główne funkcje	3
1.2	Architektura	3
2	Sitemap — ścieżki API	4
2.1	Strony (HTML)	4
2.2	API (JSON)	4
3	Schemat bazy danych	5
3.1	Relacje	6
4	Opis testów	6
5	Instrukcja uruchomienia	7
5.1	Wymagania	7
5.2	Konfiguracja (zmienne środowiskowe)	7
5.3	Uruchomienie GROBID	7
5.4	Uruchomienie aplikacji	7
5.5	Włączenie logowania (opcjonalnie)	7
5.6	Migracja nazw plików (jednorazowo)	7
5.7	Uruchomienie testów	8
6	Zrzuty ekranu	8

1 Opis funkcjonalności

CiteScout to narzędzie do **odnajdywania prac powiązanych** (related works) przez śledzenie cytowań. Użytkownik dodaje interesujące go prace naukowe (pliki PDF), a aplikacja automatycznie wydobywa z nich metadane oraz bibliografię i pomaga odkrywać kolejne prace warte przeczytania.

Geneza: zadanie od promotora — „znajdź prace powiązane; możesz sprawdzić prace, które cytujesz w swojej pracy, oraz prace, które cytują twoją”.

1.1 Główne funkcje

- **Dodawanie prac** — przez upload pliku PDF lub podanie adresu URL. Plik jest walidowany (nagłówek %PDF-), a pobieranie z URL jest zabezpieczone przed SSRF (tylko http/https, blokada adresów nie-publicznych).
- **Ekstrakcja metadanych i cytowań** — serwer GROBID parsuje PDF i zwraca tytuł, autorów, rok, abstrakt, DOI pracy oraz listę pozycji bibliograficznych wraz z miejscami (zdaniami) ich wystąpienia w tekście i numerem strony.
- **Nazewnictwo plików wg DOI** — zapisany PDF jest nazywany bezpiecznie dla systemu plików formą DOI (fallback <id>.pdf). Dla istniejących baz dostępny jest skrypt migracyjny.
- **Deduplikacja po hashu** — identyczny plik (po SHA-256 treści) nie jest archiwizowany dwa razy; upload duplikatu przekierowuje do istniejącej pracy.
- **Rozwiązywanie DOI cytowań** — przycisk „find” zleca w tle wyszukanie DOI danej pozycji bibliograficznej (model Gemini z Google Search grounding); status jest odpytywany przez HTMX, bez przeładowania strony.
- **Tagowanie prac** — dowolne etykiety przypisywane do pracy.
- **Filtrowanie i listowanie** — lista prac z filtrami po autorze, tagu oraz treści (tytuł, abstrakt, tekst cytowań); klikalne listy autorów i tagów.
- **Discovery „cited by”** — dla pracy z DOI: dociągnięcie na życzenie listy prac, które ją cytują (OpenAlex). Realizuje wprost polecenie promotora.
- **Uwierzytelnianie** — opcjonalne logowanie jednym hasłem (SHA-256), sesja w cookie. Zaimplementowane wyłącznie na bibliotece standardowej.

1.2 Architektura

Aplikacja napisana jest w języku **Go** z użyciem wyłącznie biblioteki standardowej dla warstwy HTTP (net/http, html/template). Trwałość zapewnia **SQLite** (sterownik mattn/go-sqlite3), pliki PDF leżą na dysku w konfigurowalnym katalogu (docelowo zamontowany Google Drive). Front to lekki HTMX wstrzykiwany do szablonów.

Moduł	Odpowiedzialność
modules/repository	Warstwa danych (PaperArchive) oraz cała logika HTTP i szablony HTML/HTMX.
modules/grobid	Klient serwera GROBID — ekstrakcja nagłówka i bibliografii z PDF (TEI/XML).
modules/doi	Klient Gemini (REST) z Google Search — wyszukiwanie DOI po tytule.
modules/openalex	Klient OpenAlex — prace cytujące daną pracę („cited by”).

Moduł	Odpowiedzialność
modules/auth	Logowanie hasłem, sesje w cookie, middleware ochrony tras.
main.go	Złożenie zależności z konfiguracji (zmienne środowiskowe) i start serwera.

2 Sitemap — ścieżki API

Serwer nasłuchuje na porcie 8080. Strony zwracają HTML, API zwraca JSON (o ile nie zaznaczono inaczej).

2.1 Strony (HTML)

Metoda	Ścieżka	Opis
GET	/	Przekierowanie na listę prac.
GET	/papers/	Lista prac; filtry ?q=, ?author=, ?tag=.
GET	/new	Formularz dodania pracy (plik lub URL).
GET	/papers/{id}	Strona pracy: metadane, tagi, „cited by”, tabela cytowań.
GET	/login	Formularz logowania (gdy auth włączony).
POST	/login	Weryfikacja hasła, ustawienie cookie sesji.
GET	/logout	Wylogowanie i kasowanie sesji.

2.2 API (JSON)

Metoda	Ścieżka	Opis
GET	/api/papers/	Lista prac; przyjmuje filtry q/author/tag.
POST	/api/papers/	Dodanie pracy (multipart: pole pdf lub url).
GET	/api/papers/{id}	Metadane pojedynczej pracy.
PUT	/api/papers/{id}	Aktualizacja metadanych (JSON).
DELETE	/api/papers/{id}	Usunięcie pracy wraz z plikiem PDF i cytowaniami.
GET	/api/papers/{id}/view	Podgląd PDF w przeglądarce (inline).
GET	/api/papers/{id}/download	Pobranie PDF (attachment).
GET	/api/papers/{id}/citations	Cytowania wydobyte z pracy.
POST	/api/papers/{id}/tags	Zastąpienie tagów (pole tags, po przecinku).
GET	/api/papers/{id}/citing-works	Prace cytujące (OpenAlex, na życzenie).

Metoda	Ścieżka	Opis
POST	/api/citations/{id}/resolve-doi	Zlecenie wyszukania DOI cytowania (Gemini).
POST	/api/citations/{id}/doi	Ręczne ustawienie (doi) lub usunięcie DOI.
GET	/api/citations/{id}/doi-cell	Fragment HTMX komórki DOI (polling statusu).

3 Schemat bazy danych

Baza SQLite (archive.db) zawiera cztery tabele. Schemat tworzony jest idempotentnie przy starcie (CREATE TABLE IF NOT EXISTS), a brakujące kolumny (np. hash) są dodawane migracją ALTER TABLE.

```
CREATE TABLE papers (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  title       TEXT,
  author      TEXT,          -- autorzy złączeni przecinkami
  year        INTEGER,
  abstract    TEXT,
  doi         TEXT,          -- DOI samej pracy (jeśli wykryte)
  hash        TEXT,          -- SHA-256 treści PDF (deduplikacja)
  filename    TEXT           -- nazwa pliku PDF na dysku
);

CREATE TABLE citations (
  id           INTEGER PRIMARY KEY AUTOINCREMENT,
  paper_id     INTEGER,       -- FK -> papers(id)
  citation_number INTEGER,    -- pozycja w bibliografii (1-based)
  citation_title TEXT,
  citation_text TEXT,         -- surowy tekst pozycji bibliograficznej
  doi          TEXT,          -- DOI cytowanej pracy (jeśli rozwiązane)
  FOREIGN KEY(paper_id) REFERENCES papers(id)
);

CREATE TABLE citation_occurrences (
  id           INTEGER PRIMARY KEY AUTOINCREMENT,
  citation_id   INTEGER,       -- FK -> citations(id)
  occurrence_text TEXT,        -- zdanie, w którym cytowanie wystąpiło
  page         INTEGER,        -- numer strony
  FOREIGN KEY(citation_id) REFERENCES citations(id)
);

CREATE TABLE paper_tags (
  id           INTEGER PRIMARY KEY AUTOINCREMENT,
  paper_id     INTEGER NOT NULL, -- FK -> papers(id)
  tag          TEXT NOT NULL,
  FOREIGN KEY(paper_id) REFERENCES papers(id)
);

CREATE UNIQUE INDEX idx_paper_tags_unique ON paper_tags(paper_id, tag);
```

3.1 Relacje

- papers 1 — N citations (praca ma wiele cytowań),
- citations 1 — N citation_occurrences (cytowanie ma wiele wystąpień w tekście),
- papers 1 — N paper_tags (praca ma wiele tagów; para (paper_id, tag) jest unikalna).

4 Opis testów

Testy uruchamia się przez `make test` (czyli `go test ./...`). Każdy pakiet ma testy jednostkowe lub funkcjonalne; te ostatnie korzystają z `net/http/httpptest` oraz baz SQLite w katalogu tymczasowym (`t.TempDir()`) i atrap (fakes) dla GROBID oraz OpenAlex.

Test	Sprawdzany wymóg
<code>TestUploadAndListPaper</code>	Upload PDF tworzy pracę z metadanami i pojawia się na liście.
<code>TestUploadDeduplicatesByHash</code>	Dwukrotny upload tego samego pliku daje jedną pracę (dedup).
<code>TestTagsAndFiltering</code>	Ustawienie tagów oraz filtrowanie po tagu, autorze i treści.
<code>TestCitingWorksFragment</code>	Discovery „cited by” renderuje listę prac cytujących.
<code>TestValidateFetchURLRejectsNonPublic</code>	Odrzucenie URL-i SSRF (loopback, prywatne, zły schemat).
<code>TestFindByHashDetectsDuplicate</code>	Wyszukiwanie pracy po hashu; pusty hash nie pasuje.
<code>TestEnsureColumnIsIdempotentAndMigrates</code>	Migracja kolumny hash na starej bazie.
<code>TestPDFFilename</code>	Generowanie bezpiecznej nazwy pliku z DOI.
<code>TestDOIQueueFillsMissingDOIs</code>	Kolejka w tle uzupełnia brakujące DOI cytowań.
<code>TestDisabledAuthPassesThrough</code>	Brak hasła = brak ochrony (przejsie).
<code>TestMiddlewareBlocksUnauthenticated</code>	Bez sesji: API → 401, strony → redirect na / login.
<code>TestLoginFlowGrantsAccess</code>	Pełny cykl: złe hasło, logowanie, dostęp, wylogowanie.
<code>TestCitingWorks</code>	Klient OpenAlex: rozwiązanie pracy i lista cytujących (mock).
<code>TestNormalizeDOI</code>	Normalizacja DOI (usunięcie prefiksu URL, lowercase).
<code>TestExtractDOI</code>	Wyciągnięcie DOI z odpowiedzi modelu.
<code>TestParseHeader / TestParseFulltext*</code>	Parsowanie TEI/XML z GROBID (nagłówki, cytowania, wystąpienia).

Testy uruchamiane są również z detektorem wyścigów (`go test -race ./...`) ze względu na współbieżną kolejkę DOI i magazyn sesji.

5 Instrukcja uruchomienia

5.1 Wymagania

- Go (wersja zgodna z `go.mod`),
- działający serwer GROBID (np. przez Docker Compose),
- opcjonalnie: klucz API Gemini (DOI cytowań), `sqlite3` CLI (skrypt migracji).

5.2 Konfiguracja (zmiennne środowiskowe)

Zmienne czytane są z pliku `.env` (nie nadpisują istniejących zmiennych powłoki).

Zmienna	Znaczenie
PAPERS_DIR	Katalog na pliki PDF (domyślnie <code>data/papers</code>).
GROBID_URL	Adres serwera GROBID (domyślnie <code>http://localhost:8070</code>).
GROBID_CONSOLIDATE	1 = rozwiązuj DOI cytowań w GROBID przez CrossRef (wolniej).
GEMINI_API_KEY	Włącza wyszukiwanie DOI cytowań (web search).
GEMINI_MODEL	Model Gemini (domyślnie <code>gemini-2.5-flash</code>).
OPENALEX_MAILTO	E-mail kontaktowy do „polite pool” OpenAlex.
AUTH_PASSWORD_HASH	SHA-256 hex hasła logowania; ustawione = wymóg logowania.

5.3 Uruchomienie GROBID

```
docker compose up -d grobid
```

5.4 Uruchomienie aplikacji

```
# kompilacja i start
go run main.go
# lub: make compile && ./dist/cite-scout

# aplikacja wystartuje na http://localhost:8080
# formularz dodawania: http://localhost:8080/new
```

5.5 Włączenie logowania (opcjonalnie)

```
# wygeneruj hash hasła i wstaw do .env jako AUTH_PASSWORD_HASH
make hash-password PASSWORD=tajnehasło
```

5.6 Migracja nazw plików (jednorazowo)

```
# podgląd zmian bez modyfikacji
DRY_RUN=1 make migrate-filenames
# właściwa migracja
make migrate-filenames
```

5.7 Uruchomienie testów

```
make test
# lub z detektorem wyścigów:
go test -race ./...
```

6 Zrzuty ekranu

< > × localhost:8080/new

[← All papers](#)

Archive a scientific paper

Add a PDF by uploading a file or giving a link to one. Its title, author, year and abstract are filled in automa

PDF file

1-s2.0-S037...70-main.pdf

— or —

PDF URL

Rysunek 1: Dodawanie nowego pliku PDF lub URL pracy naukowej.

Scientific paper archive

[+ Add paper](#)

Authors: [Achour Mahrane](#) [David Thomas](#) [Hamza Mellani](#) [Khaled Tifoura](#) [Mahmoud Elkazaz](#) [Mark Sumner](#) [Richard Davies](#) [Seksak Pholboon](#)

Tags: [HEMS](#)

Title	Author	Year	DOI	Tags	
Optimization based Real-Time Home Energy Management in the Presence of Renewable Energy and Battery Energy Storage	Mahmoud Elkazaz, Mark Sumner, Richard Davies, Seksak Pholboon, David Thomas	—	—	HEMS	view delete

Rysunek 2: Lista prac z paskiem filtrów (autor / tag / treść).

localhost:8080/papers/2

All papers

Home energy management system based on applied real-time load scheduling for self-consumption enhancement

Author
Khaled Tifoura, Hamza Meliani, Achour Mahrane

Year
2025

DOI
10.1016/j.enbuild.2025.116107

PDF
view · download

Tags
comma, separated, tags save

Abstract

In the era of sustainable energy, solar home systems (SHS) play a pivotal role in decentralized power generation. However, optimal solar energy utilization remains challenging due to variable generation and demand patterns. This work introduces an applied approach to load scheduling in SHS, focusing on real-time optimization of photovoltaic (PV) energy consumption using particle swarm optimization (PSO) to minimize prosumer grid exchange and achieve a high self-consumption rate. The study employs real-time scheduling algorithm of house appliances, aiming to dynamically adjust their operation time to match solar generation and household demand, by exploiting real-time data and forecasted PV power production. The proposed strategy was implemented on a Raspberry Pi in solar house demonstrator, using fore PV production profiles and measured household consumption. In addition, a low-cost Wi-Fi control device has been developed to automatically switch shiftable appliances. The results obtained demonstrate the effectiveness of the proposed approach by minimizing the grid exp power by more than 78% and enhancing the self-consumption rate by up to 42%. The findings highlight the potential of real-time load scheduling as a viable solution for improving the efficiency and sustainability of solar home systems without affecting user comfort, contributi advance intelligent energy management in home areas.

Cited by

Find works that cite this paper

Citations (47)			
No	DOI	Appears on	Title
1	10.1787/827374af-en 10.1787/827374af-en save remove	- According to the International Energy Agency (IEA), world energy demand increased by more than 25 % between 2010 and 2020 [1] inducing an increase in greenhouse gas (GHG) emissions into the atmosphere. (p.1) % than in scenario (1). (p.15)	International Energy Agency (IEA), "World Energy Outlook 2023," 2023, Paris. [Online]. Available: https://www.iea.org/reports/world-energy-outlook-2023.
2	10.1007/978-3-031-25984-5_300412 10.1007/978-3-031-25984-5_300412 save remove	The main one is the Paris Climate Agreement, which aims to reduce global warming to less than 1.5 °C by lowering greenhouse gas emissions [2]. (p.1)	"COP21: COP21 Paris France Sustainable Innovation Forum."
3	find 10.xxxxxx set	The residential sector represents a significant part of the total energy consumption in the world, which is around 29 % [3]. (p.1)	International Energy Agency (IEA), "Energy Efficiency 2023," 2023, Paris. [Online]. Available: https://www.iea.org/reports/energy-efficiency-2023.
.	10.1002/tee.24024 10.1002/tee.24024	To achieve this aim, the experts have proposed two approaches: a passive and active approach [4,5] and	B. Li, W. Cao, T. Tang, B. Qi, J. Zhao, C. Liu, QoS-based bi-level demand response for data center to

Rysunek 3: Strona pojedynczej pracy: metadane, tagi i tabela cytowań.